

Handover - testing the copilot against a local MCP

For: Aditya. From: Sanujit. Date: 14 Sep 2026.

Branches: app feat/copilot-finetune (PR #59, draft, base copilot-upgrade) | MCP local/mcp-integration-main (not a PR - see sec.6).

The rule we have been working to: **a live failure -> its reason from the log -> a fix reproduced through the copilot against the MCP running locally -> then a PR.** Unit tests alone do not count.

Every prompt run so far is quoted verbatim in docs/copilot/PROMPT-LIBRARY.md; the prompt set to run is docs/copilot/PROMPT-BATTERY.md (also as PDF). Read HANDOFF-model-proposes.md sec.8 first for the one-page explanation of the pipeline and what changed.

1. Bring the loop up (three processes + the app)

#	What	Where / how	Check
1	Postgres for the Sign Service	Docker container vanna-sign-pg on 5444	docker ps shows it
2	Sign Service (local - the deployed one is IAM-restricted, a laptop gets 403)	vanna-mcp/sign-service: npm run api on 8787. .env keys: PORT DATABASE_URL SIGN_SERVICE_SECRET SECRETS_SOURCE=gcp GCP_PROJECT_ID IDENTITY_ENFORCE PRIVY_APP_ID WORKOS_ISSUER WORKOS_AUDIENCE SOROBAN_RPC_URL NETWORK_PASSPHRASE. Needs gcloud auth application-default login.	port 8787 listening
3	The wallet ? identity binding for your test user	npm run backfill:identity-bindings (src/scripts/backfill-identity-bindings.ts) with the Privy DID -> G-address of the wallet you sign in with	vanna_wallet list_bindings shows it
4	MCP server	vanna-mcp on branch local/mcp-integration-main, streamable-http 127.0.0.1:8765, the deployed auth settings: AUTH_ENABLED=true OAUTH_ISSUER=https://sensitive-silk-47-staging.authkit.app/ OAUTH_JWKS_URI=.../oauth2/jwks OAUTH_AUDIENCE=https://mcp.vanna.finance/mcp OAUTH_M2M_AUDIENCE=client_01KX5H81JH2HWD2DHKYFYFXNS2 EXTERNAL_BASE_URL=http://127.0.0.1:8765 SIGN_SERVICE_URL=http://127.0.0.1:8787 S2S_AUTH_MODE=shared_secret SIGN_SERVICE_SECRET=<same as sign-service/.env> FORWARD_USER_ASSERTION=true MCP_TOOL_SURFACE=composites; run python -m mcp_server.main	tools/list returns 18 tools, vanna_earn_market lists preview
5	The app	mercury-stellar-backend on feat/copilot-finetune, .env.local: MCP_MODE=live MCP_BASE_URL=http://127.0.0.1:8765/mcp (+ the existing WORKOS_M2M_*, GOOGLE_CLOUD_PROJECT=vanna-mcp, VERTEX_MODEL=gemini-3.6-flash, COPILOT_RESEARCH_SECRET, service-account JSON). npm run dev on 3000. Sign in with Privy on /copilot.	first prompt returns a card

The app's M2M client (client_01KXBNHSTPDZZ90370X7JEQ7HS) is in the MCP's grandfathered first-party write list, so writes work without vanna:write in the token. When you point the app back at the hosted MCP, set MCP_BASE_URL=https://mcp.vanna.finance/mcp (nothing else changes).

2. What the copilot now does that touches the MCP

- Every plan is sized in code from reads (plan.ts) - the model only names shapes and sizing

words. Amounts come from wallet_balances (spendable!), account_debt, account_collateral, earn_position, asset_price, earn_market, blend_markets. If a read is wrong, the card is wrong.

- Stated writes ("lend 1 xlm") go through the same sizer since today - they used to compile straight to a step with no read. So a plain "repay 1 XLM" now makes 3-4 reads before offering.
- Propose-time simulation calls vanna_margin_status | preview and vanna_earn_market | preview for every step the chain can be asked about, at research time and again at Prepare. A preview that says allowed:false removes the option with the MCP's own reason. An MCP without the action (invalid_input), a timeout or an error never blocks - the card says "not simulated".
- Spendable XLM: the copilot uses the wallet read's spendable (and min_balance in the refusal copy). Without PR #3 it falls back to balance - fee_reserve_xlm, which does not know the chain minimum - a 3.94 XLM wallet still gets "lend 1 XLM" offered and the contract refuses it (HostError #10). That case is only fully fixed on hosted once #3 deploys.
- Aquarius LP reads resolve the pool through the router (PR #4). Without it, every LP prompt reports "no pool".
- Repay is executed as deposit -> repay (vanna_repay draws from the smart account); a repay the account already covers stays one step.

3. The prompts to run, and what a correct card looks like

Run one prompt at a time and hard-reload after any code change (stale chunks otherwise). Do not run vitest/tsc in the same machine while a prompt is in flight - it stalls the dev server.

Prompt	Expect
lend 1 xlm to earn (wallet with only the minimum balance)	refused at planning time: "3.94 XLM is held, but 3.5 XLM is the chain's minimum balance and 0.5 XLM is the fee reserve - nothing is spendable". Nothing to approve. (Needs #3.)
lend 25% of xlm that i hold and also repay 25% of xlm debt	3 steps: Lend 25 % of spendable -> Deposit 25 % of debt -> Repay it; "Leaves ... of debt"; a "Simulated against the protocol ..." line under the steps
what are the debt tokens currently i am holding	"Debt: XLM 14,113.4967 (\$...), BLUSDC 772 (\$...); total \$" - rows, not just the total
repay 1 XLM	one step if the account holds it, else Deposit 1 -> Repay 1; "Approve to run"
Deposit 10000 XLM as collateral and deploy it in the Blend farm, keep HF above 1.15	deposit -> supply_blend, both settle without a click in between
use my AqUSDC sitting in Earn as collateral, keep HF above 1.15	redeem -> deposit, both settle
put my XLM and USDC into the Aquarius XLM/USDC LP	pool named (via #4); LP refused as not executable; nothing substituted
how much xlm can i withdraw ??	with no floor stated: "... tell me the number; at the line itself up to N XLM of the M posted could come out" - a figure, not a question back. On the test account this is refused earlier by the disagreement rule (sec.4).

Prompt	Expect
withdraw all my balance from margin acc make sure to keep my HF > 2.5	sized to G - 2.5[D, capped by what is posted - same sec.4 caveat
invest into earn pool where i can get the best/good returns	ranked Earn options with rates, an "Idle in the wallet: ..." line

Anything else: pick from the battery (sections B-O). Record every run in PROMPT-LIBRARY.md in the existing entry format - verbatim card text, log lines, cause, and cause: undetermined rather than a guess.

4. Conditions on the test account that are NOT defects

- ~883 XLM sits in the margin account unposted. The Margin page counts it, the liquidation engine does not, so the two snapshots disagree and the copilot **refuses every borrow and every withdraw** with "the Margin page and the liquidation engine disagree on your position" - by design (owner rule). Deposits and repays still size. To test borrows/withdraws, post that XLM as collateral (or use an account where the two agree).
- Wallet XLM near the minimum balance (3.94 XLM): everything wallet-funded is refused; that is the correct answer. Faucet if you want funded runs.
- Blend XLM at ~168 % APR / 90 % utilisation is real testnet data, not a display bug.

5. Things that look like copilot bugs and are not

Symptom	What it is	How to tell
"The investigation ran out of time" / "language model was unavailable" in ~10 s	outbound network: investigation model failed { fetch failed }, read ECONNRESET, computeMarginSnapshot timed out after 12000ms in the dev log	grep the dev-server log for the request id
Card says Checked in 9s 1m 59s this device	the server took 9 s; the request sat in the browser ~110 s before it was sent. Open DevTools -> Network -> investigate -> Timing. "Stalled" = Chrome's 6-connections-per-origin limit - close other localhost:3000 tabs. Still open; not reproduced from the server side.	Timing tab
Sending your request for minutes, nothing in the server log	same as above - the request never arrived	no investigate start line
ChunkLoadError / "Preparing your session" after a code change	stale HMR chunks	hard-reload
Everything slow at once	disk was at 0 bytes free on 14 Sep (.next cache alone was 4.6 GB)	df -h

6. MCP side - what is where

Change	Branch / PR	Status
spendable + min_balance on the wallet read (Horizon reserve fields)	feat/mcp-wallet-spendable -> PR #3	verified through the copilot locally (deposit 202.496924 XLM settled). Needs merge + deploy. Blocks the "lend 1 xlm" fix on hosted.

Change	Branch / PR	Status
Aquarius pool by router / token contract, concurrent pages	feat/mcp-aquarius-pool-by-router -> PR #4 (rebased on b5398d1)	verified locally (pool named). Needs merge + deploy.
Cache a failed identity-token mint (12 s per call otherwise)	fix/mcp-identity-token-mint-cache (was #5, closed)	local-loop enabler only - the hosted Sign Service is reachable from Cloud Run, so hosted never hits it. Keep on the local branch.
local/mcp-integration-main	= origin/main (b5398d1) + the three above	what the loop runs today; 600 pytest pass. Rebuild it the same way after main moves.

Hosted MCP is main @ b5398d1 (18 composite tools, preview present). The hosted Earn preview accepts holder but does not check the holder's balance - the sizer's spendable check is the real guard, hence #3.

7. Open, in order

1. Merge + deploy #3 then #4; point the app at hosted; re-run sec.3 rows 1 and 7.
2. The browser-side stall (sec.5 row 2) - needs the Timing tab from a stalled run.
3. The "0.00 % APR" chip on a repay option (cosmetic; the reply already says "Repays N XLM ...").
4. blend_withdraw, swap, deploy_to_blend are not in WORKFLOW_OPS yet (each: a row in the op-flow table + allowlist + a sizer branch; the shape matrix then covers it by itself).
5. UI/UX pass (conversation history instead of "clear session", new chat, reload state) - parked.

8. Where to look

- Dev-server log: every copilot request logs investigate start {request_id} -> investigation phase ...
-> [mcp-client] call { tool, ms, keys } -> investigate done. The keys list is the first thing to check when a read "succeeds" but the card is wrong.
- MCP log: one POST /mcp per call; Soroban/Horizon calls as httpx lines.
- Tests: npx vitest run (1839; the shape matrix alone is ~25 s). npx tsc --noEmit -p .
- Contracts: Protocol_V1_Soroban - RiskEngineContract/src/risk_engine.rs for how positions are valued (Blend receipts count at underlying x oracle; that is why a Blend supply is health-neutral).